



Deep learning. Redes neuronales convolucionales aplicadas a la detección de la malaria

Olaya Barcelona Fernández

Trabajo de fin de grado en física

Universidad de Zaragoza

Director

Ricardo López Ruiz

Julio 2024

Índice

1. Introducción y objetivos

2. Redes Neuronales 1

2.1. Estructura básica de las redes neuronales 1

2.1.1. Unidad de procesamiento 1

2.1.2. Tipos de funciones de activación 2

2.1.3. Topología de redes neuronales 4

2.2. Optimización y función de pérdida 4

2.2.1. Función de pérdida 4

2.2.2. Optimización y descenso de gradiente 5

2.3. Entrenar una red neuronal con retropropagación 6

3. Redes neuronales convolucionales 9

3.1. La estructura básica de las redes convolucionales 9

3.1.1. Convolución 10

3.1.2. Capa *ReLU* 12

3.1.3. *Pooling* 12

3.1.4. Capa densa 13

3.1.5. Intercalación de capas en una *CNN* 14

3.2. Hiperparámetros 15

3.2.1. *Padding* 15

3.2.2. <i>Strides</i>	15
3.3. Sobreajuste y regularización	16
3.3.1. <i>Data augmentation</i>	17
3.3.2. <i>Dropout</i>	17
4. Aplicación a la detección de malaria	18
4.1. Planteamiento del problema	18
4.2. <i>Dataset</i>	19
4.3. Arquitectura de la red	20
4.4. Entrenamiento de la red	21
4.5. Resultados	22
4.6. Conclusiones	23
Bibliografía	25
Anexos	26
A. Código Fuente	27
B. Entrenamiento de la CNN	32

Capítulo 1

Introducción y objetivos

Las redes neuronales artificiales son un modelo computacional de *Machine Learning* (*ML*)¹ que simula el mecanismo de aprendizaje de organismos biológicos. El primer modelo simplificado de una neurona biológica lo presentaron McCulloch y Pitts en 1943, pero fue en los años ochenta tras algunos resultados teóricos importantes, cuando resurgió el interés por las redes neuronales.

Los seres humanos y los ordenadores desempeñamos tareas diferentes con buen desempeño. Los cálculos matemáticos o problemas de lógica, son el tipo de tareas que puede realizar una computadora fácilmente, pero que al ser humano le es mucho más costoso. Por otro lado, en procesos como reconocimiento de objetos o el lenguaje, el cerebro humano superaba hasta hace poco tiempo al ordenador. Ha sido en estos últimos años cuando el *deep learning* no solo ha igualado el desempeño de las personas, si no que en algunos casos lo ha superado.

El aprendizaje profundo es un subconjunto de *ML* formado por redes neuronales de múltiples capas cuyos algoritmos están basados en el aprendizaje de múltiples características en ingentes cantidades de datos. La comparativa biológica de estos modelos es una sobresimplificación del funcionamiento del cerebro, pero ha sido útil para el diseño de estos algoritmos. Un ejemplo es el uso de redes neuronales convolucionales en el reconocimiento de imágenes. Esta arquitectura fue inspirada por los experimentos de Hubel y Wiesel [1] en 1959 en la organización de las neuronas en la corteza visual del gato.

Una de las ventajas de las redes neuronales respecto de otros algoritmos de *ML* es que tienen un mayor nivel de abstracción debido a su estructura multicapa, que les permite realizar cálculos complejos y extraer características relevantes de los datos de entrada. Además, tienen mayor flexibilidad para abordar problemas complejos ya que se pueden combinar distintos tipos de capas dependiendo del tipo de datos y se puede ajustar al límite computacional

¹ Advertencia: Los estrangerismos y las abreviaturas irán en cursiva en todo el documento.

quitando neuronas de la arquitectura.

En gran parte, la razón del éxito reciente de las redes neuronales se debe a la drástica reducción del tiempo de computación y al gran aumento y calidad de los datos disponibles. Mientras que para conjuntos de datos pequeños los algoritmos de ML tradicionales son más efectivos, en problemas en los que se posee gran cantidad de datos, *deep learning* es capaz de resolver problemas inimaginables.

En el siguiente capítulo se hará una introducción a las redes neuronales y después se profundizará en las redes neuronales convolucionales. Por último se llevará a cabo una aplicación práctica de las mismas para la detección de la malaria en células.

Capítulo 2

Redes Neuronales

2.1. Estructura básica de las redes neuronales

2.1.1. Unidad de procesamiento

Una unidad de procesamiento o neurona artificial es un componente básico de las redes neuronales artificiales, inspirado en el funcionamiento de las neuronas biológicas. Su objetivo es recibir varias entradas, procesarlas y generar una salida.

La neurona recibe valores de **entrada** o *input*, generalmente representados como $\vec{X} = [x_1, \dots, x_n]$. A cada entrada se le asocia un *peso* $\vec{W} = [w_1, \dots, w_n]$, que son números reales que expresan la importancia de las respectivas entradas en la salida de la neurona. La neurona calcula entonces una suma ponderada de las entradas, es decir, $\sum_{j=1}^n w_j \cdot x_j$, la cual es aplicada a una función llamada **función de activación**. Diferentes elecciones de funciones de activación pueden ser utilizadas para simular distintos tipos de modelos utilizados en el aprendizaje automático.

Ahora debemos introducir el **sesgo** o *bias*. Se trata de un valor constante que se agrega a la suma ponderada de las entradas antes de aplicar la función de activación. Su propósito es permitir que la función de activación se desplace hacia la izquierda o hacia la derecha, lo que ayuda a la red a ajustarse mejor a los datos.

Finalmente, tenemos que la salida de la unidad de procesamiento es la siguiente:

$$y = f(\vec{W} \cdot \vec{X} + b) = f\left(\sum_{j=1}^n w_j \cdot x_j + b\right) \quad (2.1)$$

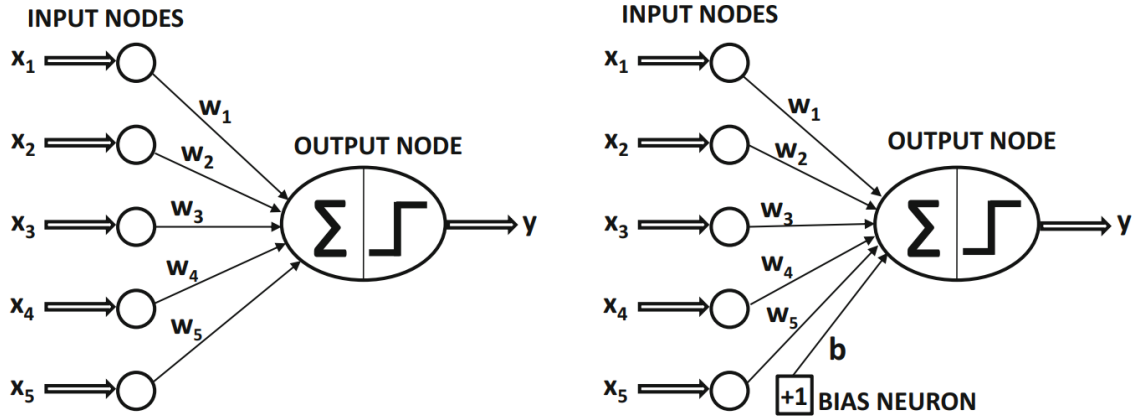


Figura 2.1: Diagrama de la estructura de una unidad de procesamiento. A la izquierda sin sesgo y a la derecha con. Fuente: *Neural Networks and Deep Learning, a Textbook* [2]

2.1.2. Tipos de funciones de activación

Las funciones de activación pueden ser lineales o de paso binario pero las más utilizadas en las en las redes neuronales artificiales son las no lineales. Estas facilitan que un modelo de red neuronal artificial se adapte a una variedad de datos y pueda diferenciar entre las salidas de manera sencilla. A continuación, se describen las funciones de activación no lineales más comunes [3] y los tipos de modelos de aprendizaje automático que pueden simular.

Función sigmoide

La función sigmoide convierte las salidas en un rango de 0 a 1, lo que es útil para tareas de clasificación binaria. Es fácil de usar y tiene todas las cualidades deseables de las funciones de activación: no linealidad, diferenciación continua, monotonía y un rango de salida establecido. Sin embargo, no está centrada en cero, lo que significa que sus valores oscilan entre 0 y 1, y no están equilibrados alrededor de cero. Esto puede llevar a que las actualizaciones de los gradientes se muevan demasiado lejos en diferentes direcciones durante el proceso de optimización, lo que dificulta el entrenamiento de la red neuronal:

$$f(x) = \frac{1}{1 + e^{-x}} \quad (2.2)$$

Función Tangente Hiperbólica (TanH)

La función TanH comprime un número de valor real al rango $[-1, 1]$ y su salida está centrada en cero. La principal ventaja de esto es que las entradas negativas serán asociadas fuertemente con valores negativos, y las entradas cercanas a cero serán transformadas casi a cero en la salida de la función TanH. Esta función también tiene el problema del gradiente

desvaneciente, pero el gradiente es más fuerte para TanH que para la sigmoide:

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (2.3)$$

ReLU (Rectified Linear Unit)

Es una de las funciones de activación más utilizada, principalmente en redes neuronales profundas como redes convolucionales. Su rango está entre 0 e infinito.

A diferencia de otras funciones de activación como la sigmoide o la tangente hiperbólica, ReLU no satura para valores grandes de entrada, lo que significa que no experimenta el problema del desvanecimiento del gradiente. Además, esto tiende a conducir a una convergencia más rápida durante el entrenamiento en comparación con funciones de activación saturadas. Otra ventaja es que es computacionalmente eficiente de calcular, ya que solo implica una operación de comparación y, en su mayoría, retorna 0 para valores negativos de entrada, lo que acelera el proceso de entrenamiento.

Un inconveniente de esta función es el llamado "problema de las neuronas muertas". Las neuronas con ReLU pueden volverse inactivas si su salida es 0 para todas las entradas durante el entrenamiento. Una vez que esto sucede, la neurona no puede activarse nuevamente, lo que puede llevar a la "muerte" de ciertas neuronas en la red. Que la salida sea 0 para entradas negativas también puede llevar a la pérdida de información útil. En algunos casos, esto puede causar problemas de propagación hacia atrás durante el entrenamiento:

$$f(x) = \max(0, x) \quad (2.4)$$

Leaky ReLU

Leaky ReLU es una variante de ReLU que evita el problema de las neuronas muertas, pero que conlleva mayor complejidad ya que añade un parámetro adicional:

$$f(x) = \max(\alpha x, x) \quad (2.5)$$

Softmax

Convierte las salidas de una red neuronal en una distribución de probabilidad sobre múltiples clases. Se utiliza en la capa de salida de una red neuronal en problemas de clasificación multiclase:

$$f(x) = \frac{e^{x_i}}{\sum_j e^{x_j}} \quad (2.6)$$

2.1.3. Topología de redes neuronales

Una red neuronal artificial suele contener más de una capa computacional, la cual está compuesta por un conjunto de unidades de procesamiento. A las capas intermedias entre la entrada y la salida se les llama capas ocultas ya que los cálculos computacionales realizados en las mismas no son visibles para el usuario.

En general, la estructura de las redes neuronales de múltiples capas puede ser de alimentación directa o recurrente, dependiendo de cómo se conectan las unidades entre sí (ver Figura 2.2). Aquí hay una distinción entre ambos [4]:

- **Redes de alimentación directa:** En este tipo de redes, el flujo de información de la entrada a la salida es estrictamente de alimentación directa. Esto quiere decir que no hay conexiones de la salida de una unidad a la entrada de una unidad en la misma capa o en capas anteriores. No hay bucles de retroalimentación que permitan que la información retroceda en la red.
- **Redes recurrentes:** Sí que contienen conexiones hacia atrás, por lo que, al contrario que en las redes de alimentación directa, las propiedades dinámicas de la red son importantes.

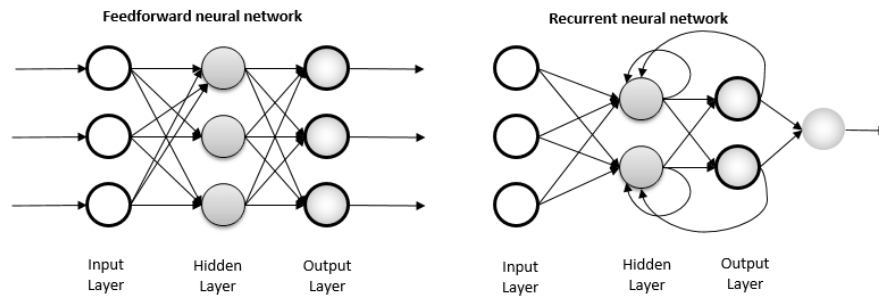


Figura 2.2: A la izquierda la estructura básica de una red de alimentación directa y a la derecha la estructura básica de una red recurrente. Fuente: *A comprehensive review for artificial neural network application to public transportation* [5]

2.2. Optimización y función de pérdida

2.2.1. Función de pérdida

En el campo del aprendizaje automático y las redes neuronales, la función de pérdida juega un papel crucial en el proceso de entrenamiento. Es fundamental para evaluar el rendimiento del modelo y guiar su optimización.

La función de pérdida, también conocida como función de costo o función objetivo, cuantifica cómo de buenas son las predicciones del modelo teniendo en cuenta los valores reales. Se utiliza para ajustar los parámetros y para evaluar la eficacia del modelo [6].

Existen varios tipos de funciones de pérdida, cada una diseñada para problemas de aprendizaje específicos. Algunas funciones comunes son la entropía cruzada binaria y el error cuadrático medio (MSE), que calcula el promedio de los errores al cuadrado entre los valores predichos y los valores reales:

$$MSE(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (2.7)$$

2.2.2. Optimización y descenso de gradiente

La optimización es el proceso de buscar los mejores valores disponibles entre un conjunto de entradas para maximizar (o minimizar) una función objetivo. En nuestro caso vamos a querer minimizar la función de pérdida.

Hay dos tipos de algoritmos de optimización, los que utilizan información de derivadas, que son rápidos y eficientes (ej. descenso de gradiente) y los que no utilizan información de derivadas porque las derivadas de la función objetivo no se pueden calcular (ej. algoritmos de evolución estratégica). En *Machine Learning* se utilizan generalmente algoritmos de optimización diferenciable y suelen ser algoritmos iterativos.

El **descenso de gradiente** es un algoritmo de optimización iterativo de primer orden que permite encontrar mínimos locales en una función diferenciable [7]. Este ajusta los parámetros del modelo iterativamente en la dirección negativa del gradiente de la función de coste. Como se puede intuir por la definición, para poder implementarlo la función de pérdida debe ser diferenciable.

Para una función $f : \mathbb{R}^n \rightarrow \mathbb{R}$ en un punto $p = (x_1, \dots, x_n)$ su gradiente se define como:

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix} \quad (2.8)$$

El gradiente descendente calcula iterativamente el siguiente punto utilizando el gradiente del punto actual. Este punto es escalado con una razón de aprendizaje $\gamma \in \mathbb{R}^+$. Se tiene entonces que el gradiente descendente utiliza la siguiente expresión:

$$p_{n+1} = p_n - \gamma \nabla f(p_n) \quad (2.9)$$

Con una razón de aprendizaje demasiado grande nunca se alcanzaría el mínimo pero una demasiado pequeña requeriría muchos pasos (tiempo computacional), y además podría estancarse en un mínimo local.

Hay diferentes maneras de utilizar el descenso de gradiente:

Descenso de gradiente *batch*

Es la técnica de optimización más sencilla ya que utiliza todos los datos de entrenamiento en cada iteración; es decir, en cada una de ellas se actualizan los parámetros con el valor medio de los gradientes de todos los ejemplos de entrenamiento disponibles.

Descenso de gradiente estocástico

Este actualiza los parámetros del modelo utilizando el gradiente calculado a partir de un único ejemplo de entrenamiento. Esto lo hace para todos los ejemplos del conjunto de datos. Como coge los ejemplos de uno en uno, la función de pérdida va a disminuir pero con fluctuaciones. Cuanto más grande sea el conjunto de datos más rápido va a converger porque hay muchas más actualizaciones que en el descenso de gradiente *batch*.

Descenso de gradiente *mini-batch*

Combina elementos del gradiente descendiente *batch* y del gradiente descendiente estocástico. Divide el conjunto de entrenamiento en subconjuntos más pequeños y actualiza los parámetros de cada subconjunto.

2.3. Entrenar una red neuronal con retropropagación

La retropropagación o *backpropagation* en inglés, es un algoritmo utilizado en las redes neuronales artificiales para ajustar los pesos de las conexiones entre las neuronas, con el fin de reducir el error en la predicción de un modelo. A continuación se desarrollarán las bases y el funcionamiento de este algoritmo [8] tan importante en *ML*.

Primero vamos a dejar clara la notación. Denotaremos w_{jk}^l al peso de la conexión de la neurona k en la capa $(l-1)$ a la neurona j de la capa l . De la misma manera, utilizaremos b_j^l

para el sesgo de la neurona j en la capa l , a_j^l al resultado de dicha neurona después de aplicar la función activación y z_j^l a la entrada ponderada (la suma ponderada de entradas antes de aplicar la función de activación). Asimismo, llamaremos σ a la función de activación. Para verlo más claramente veamos como queda la función de activación y mostramos también su forma vectorial:

$$a_j^l = \sigma\left(\sum_k w_{jk}^l a_k^{l-1} + b_j^l\right) \quad \rightarrow \quad \mathbf{a}^l = \sigma(\mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l) \quad (2.10)$$

Para que el algoritmo de retropropagación funcione debemos hacer dos suposiciones sobre la función de costo. Primeramente, que dicha función pueda ser escrita como el promedio $C = \frac{1}{n} \sum_x C_x$ de las funciones de costo C_x para cada ejemplo de entrenamiento individual. De esta manera, más adelante podremos utilizar que las derivadas parciales de la función de costo total son el promedio de las derivadas parciales de cada ejemplo individual. La segunda suposición es que la función de costo se puede escribir como una función de las salidas de la red neuronal.

Dada una función de coste C que, dado un ejemplo de entrenamiento $(\mathbf{x}, \mathbf{y})$, evalúa la similitud entre la salida obtenida de la red $\mathbf{a}^L$ para la entrada $\mathbf{x}$ y la salida esperada $\mathbf{y}$, el objetivo del algoritmo de retropropagación es calcular las derivadas parciales $\partial C / \partial w$ y $\partial C / \partial b$. Estas nos van a permitir modificar los pesos y sesgos de cada neurona de manera que se reduzca el coste C . Definimos δ_j^l como el error en la neurona j -ésima en la capa l , es decir, cuánto varía la función de coste con respecto a las entradas de dicha neurona:

$$\delta_j^l = \frac{\partial C}{\partial z_j^l} \quad (2.11)$$

A continuación, haremos una distinción entre el error en las neuronas de la capa de salida L y el error en las neuronas del resto de las capas $L-1, L-2, \dots, 1$. El error en las neuronas de la capa de salida se da por la siguiente ecuación:

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \sigma'(z_j^L) \quad \rightarrow \quad \delta^L = \nabla_a C \odot \sigma'(z^L) \quad (2.12)$$

Donde $\nabla_a C$ es el vector compuesto por las derivadas parciales de C con respecto a las diferentes activaciones de las neuronas en la capa de salida y $\odot$ el producto de Hadamard. Se trata de una operación binaria que toma dos matrices de las mismas dimensiones y produce una nueva matriz en la que cada elemento ij es el producto de los elementos ij de las dos matrices originales:

$$(A \odot B)_{ij} = A_{ij}B_{ij} \quad (2.13)$$

Siendo A y B son matrices de las mismas dimensiones. El error en las neuronas de las capas ocultas viene dado por la siguiente ecuación:

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma(z^l) \quad (2.14)$$

Donde $(w^{l+1})^T$ representa la matriz transpuesta de la matriz de pesos de la capa $l + 1$, necesaria para propagar hacia atrás los errores. Combinando las ecuaciones (2.12) y (2.14), podemos obtener el error δ_j^l en cualquier neurona perteneciente a cualquier capa en la red.

Finalmente, presentamos dos ecuaciones adicionales necesarias para poder llevar a cabo el algoritmo de manera correcta. La primera ecuación muestra cómo cambia el coste C con respecto a cualquier *bias* en la red:

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \quad (2.15)$$

y la segunda ecuación indica cómo cambia el coste C con respecto a cualquier peso en la red:

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l \quad (2.16)$$

Con estas cuatro ecuaciones fundamentales ((2.12), (2.14), (2.15) y (2.16)) ya podemos construir el algoritmo de retropropagación.

1. **Entrada x :** Establecer la activación correspondiente a^1 para la capa de entrada.
2. **Alimentación hacia delante:** Para cada $l=2,3,\dots,L$, calcular $z^l = w^l a^{l-1} + b^l$ y $a^l = \sigma(z^l)$
3. **Cálculo de errores en la capa de salida:** Calcular el vector $\delta_j^L = \frac{\partial C}{\partial a_j^L} \sigma'(z_j^L)$
4. **Retropropagación del error:** Para cada $l=L-1,L-2,\dots,2$ calcular $\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma(z^l)$
5. **Salida:** El gradiente de la función de coste está dado por las ecuaciones (2.15) y (2.16)

Este algoritmo permite obtener el gradiente de la función de coste con respecto a los parámetros del modelo para un ejemplo específico (x,y) . En la práctica, es común combinar el algoritmo de retropropagación con métodos de aprendizaje como el descenso del gradiente.

Capítulo 3

Redes neuronales convolucionales

Las redes convolucionales o *CNNs*, son un tipo de red neuronal diseñada para procesar datos con una topología en forma de cuadrícula, como por ejemplo una imagen en 2D. Este tipo de datos presentan dependencias espaciales, ya que en una imagen, los píxeles adyacentes suelen tener valores de color similares. Por ejemplo, en una fotografía de un césped, los píxeles cercanos entre sí probablemente sean todos tonos de verde semejantes. La mayoría de aplicaciones de redes convolucionales se centran en datos de imagen, que es con lo que vamos a trabajar en este capítulo, aunque se pueden utilizar para todo tipo de datos.

El nombre de este tipo de redes indica que la emplean una operación matemática llamada **convolución**, que es un tipo especializado de operación lineal. Las redes convolucionales son simplemente redes neuronales que utilizan convolución en lugar de la multiplicación de matrices general en al menos una de sus capas.

3.1. La estructura básica de las redes convolucionales

La red neuronal convolucional funciona de manera muy similar a una red neuronal tradicional de alimentación directa, excepto que las operaciones en sus capas están organizadas espacialmente con conexiones dispersas entre capas. Los tres tipos de operaciones más presentes en una red neuronal convolucional son la convolución, el *pooling* (agrupamiento) la capa de activación *ReLU* y la capa densa. A continuación, describiremos cada uno de los diferentes tipos de operaciones y capas, así como la forma típica en la que estas se intercalan en una red neuronal convolucional.

Cada capa en la red convolucional es una estructura de malla tridimensional, que tiene altura, anchura y profundidad. La palabra "profundidad" (cuando se usa en el contexto de una sola capa) se refiere al número de canales en cada capa, como el número de canales de color

primarios (por ejemplo, azul, verde y rojo) en la imagen de entrada o el número de mapas de características en las capas ocultas.

Los datos de entrada de una red neuronal convolucional están organizados en una malla 2D, donde a los valores de los puntos individuales de la malla se les llama píxeles. En el caso de los píxeles en el esquema de colores RGB, cada píxel tiene una intensidad asociada para los tres colores primarios (rojo, verde y azul). Así, el tamaño de la entrada es $L_q \times B_q \times d_q$, siendo los dos primeros términos las dimensiones espaciales, L_q la altura y B_q la anchura, y d_q la profundidad (en este ejemplo $d_q = 3$).

Las capas ocultas tienen la misma organización 3D pero cada una de las d_q mallas 2D ya no se pueden considerar una malla de píxeles sin procesar ya que han sufrido transformaciones y ya no representan una imagen.

En la red neuronal convolucional, los parámetros están organizados en conjuntos de unidades estructurales tridimensionales, conocidas como **filtros o kernels**. El filtro suele ser cuadrado en términos de sus dimensiones espaciales, que típicamente son mucho más pequeñas que las de la capa a la que se aplica el filtro. Por otro lado, la profundidad de un filtro siempre es la misma que la de la capa a la que se aplica.

3.1.1. Convolución

Matemáticamente una convolución es una integral combinada de dos funciones que muestra cómo una función modifica la otra:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau \quad (3.1)$$

En las aplicaciones de *Machine Learning* se trata como una suma finita sobre elementos de array. Además, generalmente se utilizan convoluciones sobre más de un eje a la vez. Por ejemplo, si usamos una *imagen* I bidimensional como input, probablemente también queramos usar un *filtro* K bidimensional y la operación queda entonces:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n) \quad (3.2)$$

La operación de convolución coloca el filtro en cada posición posible de la imagen (o capa oculta) de manera que el filtro se superpone completamente con la imagen, y realiza un producto punto entre los $L_q \times B_q \times d_q$ parámetros del filtro y la malla correspondiente en el volumen de entrada (con el mismo tamaño $L_q \times B_q \times d_q$). En la *Figura 3.1* se muestra

de manera gráfica para una mayor comprensión. Esta operación en la malla de la capa q da lugar a una nueva malla en la siguiente capa, de dimensiones $L_{q+1} \times B_{q+1}$, llamada **mapa de características**. En las capas ocultas la profundidad viene dada por el número de mapas de características y tendrá tantos como filtros haya en la capa anterior.

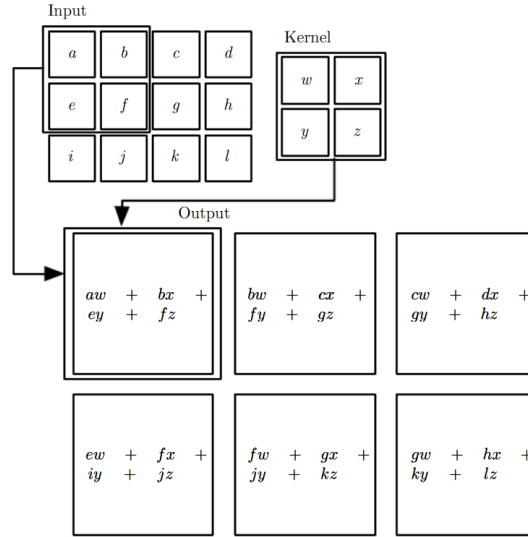


Figura 3.1: Ejemplo de convolución 2D sin volteo del filtro. La salida se restringe solo a las posiciones donde el *kernel* se encuentra completamente dentro de la imagen, lo que se llama convolución válida. Con las flechas se indica cómo se forma el elemento superior izquierdo del tensor de salida aplicando el kernel a la región superior izquierda correspondiente del tensor de entrada. Fuente: *Deep learning* pág. 330 [9]

La operación de convolución en las *CNN* sirve para detectar características en diferentes partes de una imagen. Además, estas redes están diseñadas de manera que las capas iniciales contienen filtros que detectan características simples y locales, como bordes o texturas básicas y, a medida que avanzamos en las capas, los filtros se combinan para detectar características más complejas y abstractas, como formas más definidas o características específicas de objetos.

Una propiedad de la convolución es la **invarianza de translación**, es decir, que el resultado de la convolución se traslada de manera similar cuando la entrada se traslada. En una convolución, el mismo filtro se aplica a cada parte de la imagen, por lo que los parámetros del filtro se comparten en toda la imagen y así nos aseguramos que la detección de una forma particular se procesa de la misma manera sin importar dónde aparezca en la imagen.

Otro concepto a introducir es que las *CNN* generalmente tienen **interacciones dispersas**, es decir que cada neurona en una capa está conectada solo a una pequeña región de la capa anterior, en lugar de estar conectada a todas las neuronas de la capa anterior como ocurre en una red completamente conectada. Esto se consigue haciendo el filtro más pequeño que la entrada. El **campo receptivo** se refiere al área de la imagen de entrada que afecta a una neurona en una capa determinada (ver *Figura 3.2*). A medida que avanzamos en las capas de

una *CNN*, el campo receptivo de cada neurona tiende a aumentar (3.2). Puntualizar que cada neurona en una capa convolucional de salida representa un valor en una ubicación específica de un mapa de características.

Las ventajas de de las interacciones dispersas es que se traduce en menos parámetros que aprender, lo que reduce la complejidad del modelo y el riesgo de sobreajuste. Además, calcular menos conexiones reduce el tiempo de cálculo y el uso de memoria y los filtros pueden capturar características locales como bordes, texturas y patrones pequeños.

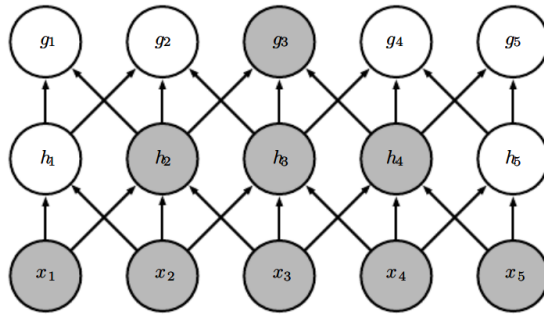


Figura 3.2: El campo receptivo de las unidades en las capas más profundas de una red convolucional es mayor que el campo receptivo de las unidades en las capas superficiales. Esto significa que, aunque las conexiones directas en una red convolucional son muy dispersas, las unidades en las capas más profundas pueden estar indirectamente conectadas a toda o la mayor parte de la imagen de entrada. Fuente: *Deep learning* pág. 332 [9]

3.1.2. Capa *ReLU*

La función de activación *ReLU* (Ec. 2.4) se suele aplicar después de una capa de convolución e introduce no linealidad en el modelo al reemplazar todos los valores negativos en la salida de la convolución por cero. Esto ayuda a la red a aprender relaciones más complejas. Consiste en aplicar la función de activación para cada uno de los $L_q \times B_q \times d_q$ valores en una capa, para crear $L_q \times B_q \times d_q$ valores umbral, lo cual no cambia las dimensiones de la capa.

Cabe señalar que el uso de la función de activación *ReLU* en este campo es reciente, pero se ha demostrado que el uso de esta función tiene grandes ventajas en cuestión de velocidad y precisión. En el diseño de *CNN*, *ReLU* ha reemplazado a todas las demás funciones de activación, por lo que vamos a usarla como la función predeterminada.

3.1.3. *Pooling*

El *pooling* es una operación que se utiliza en las redes neuronales convolucionales para reducir la dimensionalidad de los mapas de características mientras se conservan las características más importantes. Se aplica sobre cada región de tamaño $P_q \times P_q$ en cada uno

de los d_q mapas de características y produce una capa con la misma profundidad. Cada una de las regiones es evaluada independientemente.

El más común es el **max-pooling**. La operación toma un cuadrado de tamaño $P_q \times P_q$ en el mapa de características y selecciona el valor máximo en esa región. Otro tipo de pooling es el **average-pooling**, la cual toma el promedio de los valores en el cuadrado.

Si se usa un stride de 1 (ver sección 3.2.2), el tamaño de la nueva capa será $(L_q - P_q + 1) \times (B_q - P_q + 1) \times d_q$. Sin embargo, en pooling generalmente se usa un stride de $S_q > 1$, por lo que la dimensión sería $(\frac{L_q - P_q}{S_q} + 1) \times (\frac{B_q - P_q}{S_q} + 1)$, reduciendo drásticamente las dimensiones espaciales de cada mapa de características (ver Figura 3.3).

Una función importante del **pooling** es aumentar el tamaño del campo receptivo de las características mientras se reduce la huella espacial de la capa. Esto permite capturar regiones más grandes de la imagen dentro de una característica compleja en capas posteriores.

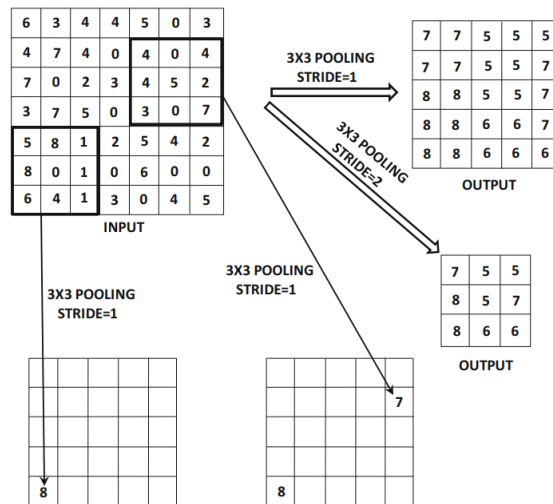


Figura 3.3: Un ejemplo de **max-pooling** de un mapa de activación de tamaño 7×7 con **stride** 1 y 2. Un **stride** de 1 crea un mapa de activación de 5×5 con elementos repetidos en gran medida debido a la maximización en regiones superpuestas. Un **stride** de 2 crea un mapa de activación de 3×3 con menos superposición. Fuente: *Neural Networks and Deep Learning, a Textbook*, pág. 337 [2]

3.1.4. Capa densa

Una capa densa, también conocida como capa totalmente conectada (*Fully Connected*), es un tipo de capa utilizada en redes neuronales artificiales donde cada neurona de la capa anterior está conectada a cada neurona de la capa actual. Esta capa permite agregar y combinar características extraídas por capas anteriores.

Suele encontrarse después de una capa de **flattening**, la cual convierte todos los mapas

de características en un único vector de una dimensión espacial. Estas capas se encuentran al final de estructura de la red, y una vez que la matriz aplanada pasa a través de una capa densa, se aplica una función de activación para clasificar las imágenes (ver *Figura 3.4*).

3.1.5. Intercalación de capas en una *CNN*

En una red neuronal convolucional, las capas convolucionales (C), las capas *ReLU* (R) y las capas de *max-pooling* (P) se intercalan de manera específica para aumentar la capacidad de la red de aprender características complejas de los datos de entrada, tal como imágenes.

En las redes neuronales tradicionales, la función de activación se combina con una transformación lineal mediante una matriz de pesos para crear la siguiente capa de activaciones. De manera similar, una *ReLU* generalmente sigue a una operación de convolución. A esta le puede seguir una capa de *max-pooling*, que es útil para reducir el número de parámetros y reducir así el costo computacional de la red.

Algún ejemplo de patrones básicos son los siguientes:

- CRCRP: Dos capas de convolución seguidas de una capa de max-pooling.
- CRCRCRP: Tres capas de convolución seguidas de una capa de max-pooling.

Estos patrones pueden repetirse varias veces y constituyen la etapa de reconocimiento de características. El final de la red corresponde a una etapa de clasificación, compuesta como ya hemos explicado en el subapartado anterior por las capas *flattering*, capa densa y una función de activación. En la *figura 3.4* se muestra la estructura de una *CNN*.

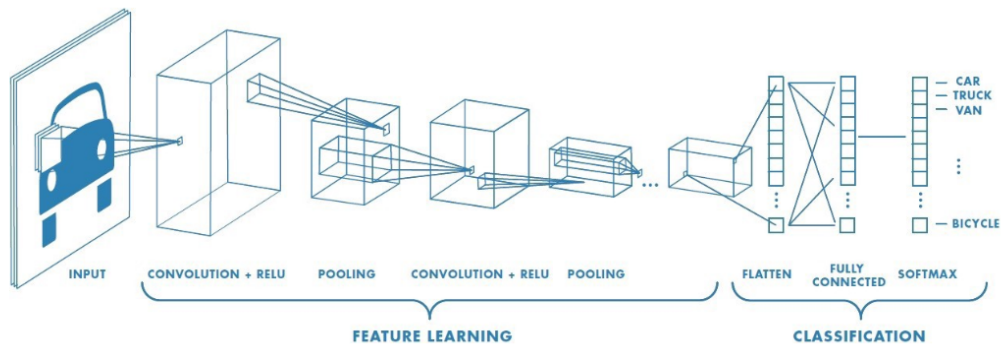


Figura 3.4: Esquema de la estructura de una *CNN*.

3.2. Hiperparámetros

Los hiperparámetros son configuraciones que se utilizan para controlar el proceso de aprendizaje y que pueden influir significativamente en el rendimiento del modelo. Algunos de ellos son el tamaño y la cantidad de filtros o el tipo de *pooling*. A continuación se explican algunos más.

3.2.1. *Padding*

Padding o relleno consiste en agregar "píxeles" alrededor de los bordes de la entrada (imagen o mapa de características) antes de aplicar el filtro de convolución (ver 3.5). Los valores agregados pueden ser ceros, algún otro valor constante o los valores que hay en los bordes. Es una técnica utilizada en las operaciones de convolución para evitar la reducción del tamaño espacial de la entrada.

Además, el *padding* también resuelve la pérdida de información importante en los bordes. Sin *padding*, los píxeles en los bordes de la entrada se procesan menos veces que los píxeles en el centro.

Existen varios tipos de *padding*:

- ***Valid padding***, así se llama cuando no se agrega ningún *padding*. El tamaño de la salida es $(L_q - F_q + 1) \times (B_q - F_q + 1)$, donde L_q es el tamaño de la entrada y F_q el tamaño del filtro.

- ***Same padding***, el cual consiste en agregar suficiente *padding* para asegurar que la salida tenga el mismo tamaño que la entrada, es decir, $\frac{F_q-1}{2}$ píxeles a cada lado del borde.

- ***Full padding***, en donde se agrega suficiente relleno a la entrada para que el filtro de convolución pueda aplicarse incluso en los bordes más extremos de la entrada, superponiéndose solo en un píxel de la imagen o mapa de características original. Se agregan $F_q - 1$ ceros a cada lado de la entrada. Esto significa que cada dimensión espacial de la entrada aumenta en $2(F_q - 1)$.

3.2.2. *Strides*

El concepto de *stride* o paso en las *CNN* se refiere a la cantidad de píxeles por los cuales se desplaza el filtro de convolución a lo largo de la entrada en cada paso de la operación de convolución.

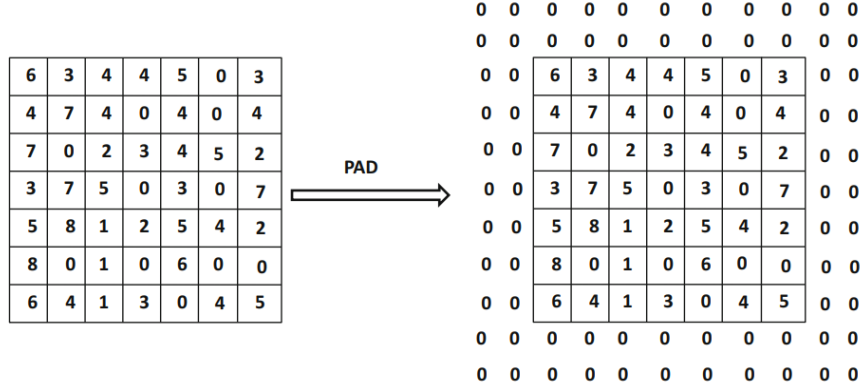


Figura 3.5: Un ejemplo de *padding* con ceros. Cada una de los d_q mapas de características de la capa q son rellenadas de esta manera. Fuente: *Neural Networks and Deep Learning, a Textbook*, pág. 333 [2]

Normalmente, cuando se realiza una convolución, el filtro se aplica en cada posición posible de la entrada. Esto corresponde a un *stride* de 1, lo que significa que el filtro se mueve un píxel a la vez. Sin embargo, no es necesario aplicar la convolución en cada posición; en su lugar, se puede ajustar el *stride* para que el filtro se mueva varios píxeles a la vez.

Cuando se usa un *stride* de S_q en la capa q , la convolución se realiza en las posiciones: $1, S_q + 1, 2S_q + 1, \dots$. El uso de un *stride* diferente de 1 reduce el tamaño espacial de la salida. El tamaño espacial de la salida se calcula como $(\frac{L_q - F_q}{S_q} + 1) \times (\frac{B_q - F_q}{S_q} + 1)$, donde L_q es la altura de la entrada, B_q es la anchura de la entrada y F_q es el tamaño del filtro.

La utilización de *strides* mayor que 1 ayuda a simplificar la computación y a reducir la dimensión espacial de la capa, lo cual es útil para gestionar la memoria y evitar el sobreajuste si la resolución espacial es innecesariamente alta.

3.3. Sobreajuste y regularización

El **sobreajuste** o *overfitting* ocurre cuando un modelo de *Machine Learning* se ajusta demasiado bien a los datos de entrenamiento y no es capaz de generalizar para los nuevos datos. Puede ocurrir cuando el modelo es demasiado complejo, los datos de entrenamiento son pocos o cuando hay demasiado ruido en los datos de entrenamiento.

Existen varias técnicas de **regularización** para mitigar el sobreajuste en las *CNN*. A continuación se explican las que se van a utilizar en la parte práctica.

3.3.1. *Data augmentation*

Data augmentation consiste en generar más datos de entrenamiento a partir de los datos disponibles. En el caso de imágenes, esto se consigue aplicando una serie de transformaciones aleatorias a la imagen como rotación o modo espejo. De esta manera se producen nuevas imágenes aunque hay que tener en cuenta que tienen que seguir teniendo sentido en la realidad. Este método ayuda a exponer el modelo a más aspectos de los datos y a generalizar mejor.

3.3.2. *Dropout*

Esta técnica consiste en desactivar un porcentaje de neuronas aleatoriamente durante el entrenamiento. Esto fuerza a la red a no depender demasiado de ninguna neurona específica y a no fijarse en detalles muy específicos.

Capítulo 4

Aplicación a la detección de malaria

4.1. Planteamiento del problema

La malaria es la infección de los glóbulos rojos por parte de una de las cinco especies del parásito *Plasmodium*, que son transmitidos a los humanos a través de picaduras de hembras de varias especies de mosquitos Anopheles. Es una enfermedad contagiosa que puede causar dificultad respiratoria, fiebre, escalofríos, diarrea o convulsiones. Estos parásitos pueden vivir en el cuerpo de una persona durante más de un año sin mostrar síntomas, lo cual es peligroso ya que la persona no recibe tratamiento y los parásitos podrían multiplicarse y causar complicaciones graves.

Según el *Informe mundial sobre la malaria 2022* [10], alrededor del 40 % de la población mundial vive en zonas afectadas por esta enfermedad, siendo el África subsahariana el área más afectada. Alrededor del 80 % de las muertes por malaria se producen en niños menores de 5 años, y la mayoría de las muertes restantes se producen en niños menores de 10 años y mujeres embarazadas. Por ello, la malaria tiene un enorme costo sanitario, social y económico.

El diagnóstico tradicional de la malaria en el laboratorio requiere de una inspección muy cuidadosa de profesionales experimentados para discernir entre glóbulos rojos sanos y glóbulos rojos infectados. Es un trabajo tedioso que lleva tiempo, y la precisión del diagnóstico puede ser impactado por la variabilidad entre observadores.

Un sistema automatizado puede ayudar con la detección temprana y precisa de malaria, y aplicaciones de técnicas de clasificación de *Machine Learning* han demostrado una precisión mayor que la clasificación manual.

En este capítulo se va a construir un modelo de visión por computadora en Python capaz de detectar malaria. Más concretamente, se aplicará la teoría presentada en los capítulos

anteriores para diseñar una red neuronal convolucional que clasifique imágenes de células como parasitadas o no infectadas.

4.2. Dataset

Los datos utilizados proceden de un repositorio del NIH [11] (Lister Hill National Center for Biomedical Communications), obtenidos de la actividad de investigación Malaria Screener. Su descarga es más rápida desde Kaggle, *Malaria Cell Images Dataset* [12]. Se trata de imágenes de glóbulos rojos infectados de dos tipos de parásitos del género *Plasmodium* (*P. vivax* and *P. falciparum*) y glóbulos rojos no infectados. Las imágenes fueron anotadas manualmente por un lector experto de portaobjetos en la Unidad de Investigación en Medicina Tropical Mahidol-Oxford en Bangkok y las células se ven de color rosa, lila o azul debido a los colorantes utilizados para su análisis.

Hay un total de 27558 imágenes de células, la mitad parasitadas y la otra mitad no infectadas (Figura 4.1). Las imágenes están en formato .png y como cada una tiene un tamaño diferente, por lo que las redimensionamos todas a 64×64 píxeles. Además, normalizamos las imágenes que están en formato RGB y las etiquetamos con "1" si corresponde a una célula parasitada y "0" si no está infectada.

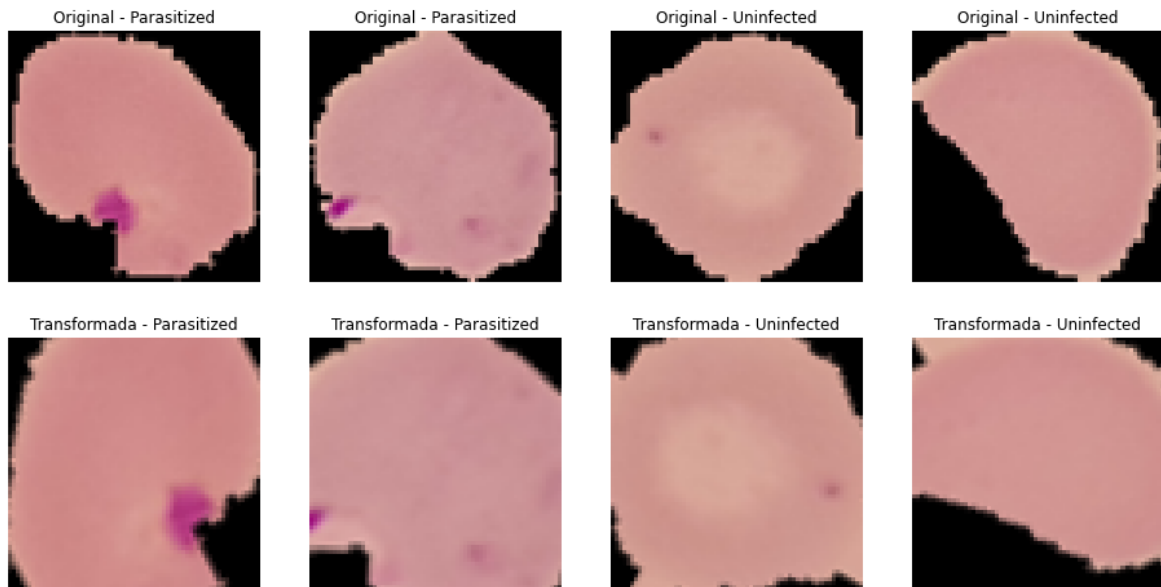


Figura 4.1: En la primera fila se muestran ejemplos de imágenes pertenecientes a la base de datos Malaria Screener. En la segunda las imágenes se corresponden a transformaciones de las imágenes de la fila superior por el método de *data augmentation*.

4.3. Arquitectura de la red

La arquitectura de la red que se ha construido consiste en primera instancia en una capa convolucional con 32 filtros de tamaño 2×2 , con un *padding* para que la salida tenga la misma dimensión espacial que la entrada (*same padding*). A continuación se añade una capa *ReLU*, seguida de otra de *Max Pooling* que actúa en cuadrados de tamaño 2×2 . Por último se añade una capa de *dropout* de coeficiente 0,2 para evitar el sobreajuste. Este patrón se repite otras dos veces más para mayor complejidad de la red.

Después de esta etapa de reconocimiento de características se añade una capa de *flattening* y otra densa con 512 neuronas en la salida. A ellas les siguen una capa *ReLU* y una de *dropout* de coeficiente 0,4. La etapa de clasificación se termina con otra capa densa, esta vez con 2 neuronas en la capa de salida, y por último una capa *Softmax* (Figura 4.2).

Además, vamos a utilizar otro método de regularización, **Data Augmentation**. Para ello primero se separa de las imágenes de entrenamiento un 10 % para la validación, ya que estas se van a mantener intactas. Las imágenes restantes se someten a una serie de transformaciones para generar nuevas imágenes de entrenamiento. Estas transformaciones incluyen volteo

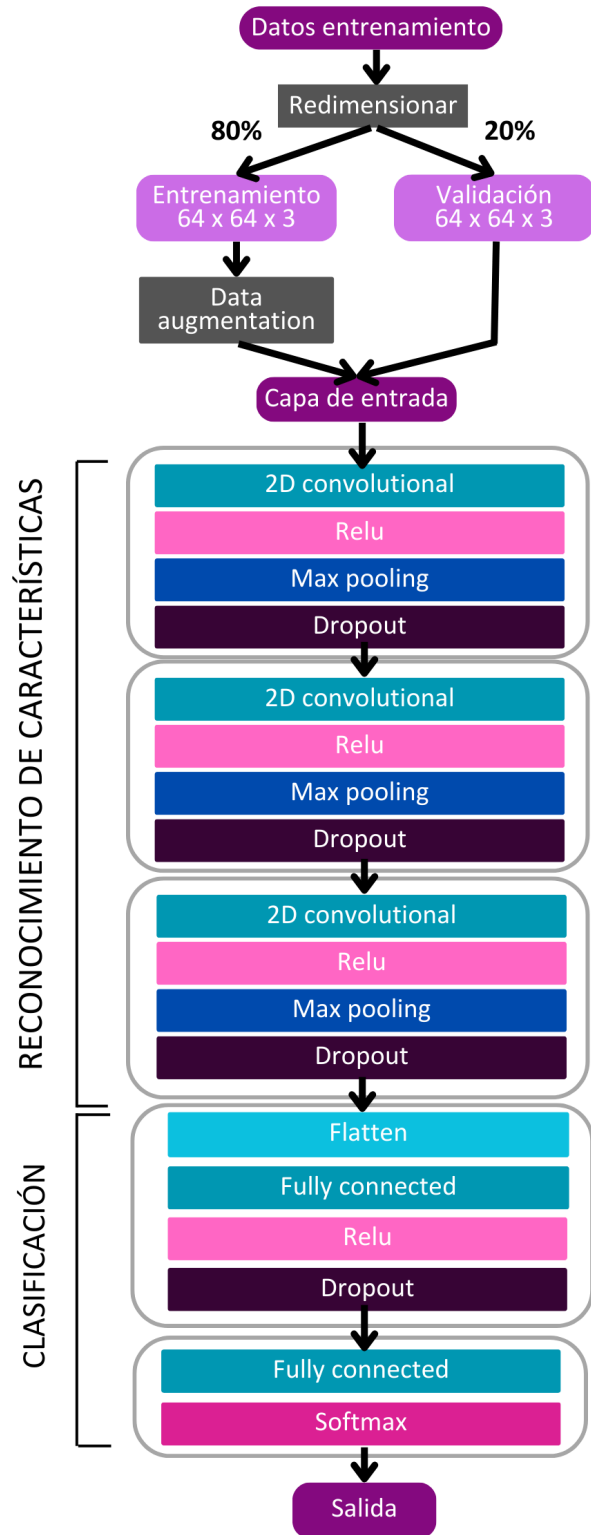


Figura 4.2: Esquema de la estructura de la CNN.

horizontal, zoom aleatorio en un rango de hasta el 70% y rotación aleatoria en un rango de hasta 30°. Si en vez de imágenes de células fuesen por ejemplo de dígitos, habría que tener más cuidado porque las imágenes carecerían de sentido. En la Figura 4.1 podemos ver algún ejemplo de las imágenes transformadas con las cuales se crea un flujo de imágenes de entrenamiento.

4.4. Entrenamiento de la red

Para el entrenamiento de la red vamos a utilizar la **Librería Keras**, una biblioteca de código abierto que trabaja con *TensorFlow* y es muy reconocida dentro del mundo de las redes neuronales.

El objetivo es obtener una *CNN* que sea capaz de identificar a través de la imagen de una célula si esta está infectada de malaria o no. La red se entrena desde cero con las imágenes del conjunto de entrenamiento (ya etiquetadas), de modo que la red resultante de este entrenamiento sea capaz de resolver el problema de clasificación.

El método de entrenamiento que se debe llevar a cabo se conoce como *aprendizaje supervisado*, ya que durante cada época, el modelo ajusta sus pesos en función de las imágenes de entrenamiento y sus etiquetas correspondientes para minimizar la función de pérdida. Las épocas o *epochs* son el número de pasadas completas a través del conjunto de datos de entrenamiento y en este caso lo definimos como 70. Asimismo, se establece el tamaño de lote o *batch size* como 32, que indica cada cuántas imágenes procesadas se actualizan los pesos. Se ha elegido ese valor ya que muchas bibliotecas de aprendizaje profundo están optimizadas para tamaños de lote de potencias de 2, lo que genera un cálculo más eficiente.

La función de pérdida empleada es la **entropía cruzada binaria** y como algoritmo de optimización utilizamos el algoritmo **Adam**. Un aspecto a destacar de este último es que utiliza una tasa de aprendizaje adaptativa para cada parámetro, lo que conlleva mayor precisión de la predicción en comparación con otros métodos de optimización.

Al final de cada época de entrenamiento, se procesan los datos del conjunto de entrenamiento que se habían separado para la validación, para obtener así una evaluación no sesgada del modelo actual con datos todavía no vistos por la red. Estos datos también pueden utilizarse para la regularización mediante la detención temprana. En este modelo se ordena que si la función de pérdida no mejora en 10 épocas (*patience*) el entrenamiento se detiene.

Después del entrenamiento, se evalúa el modelo en el conjunto de testeo para obtener

métricas de rendimiento como la precisión, la pérdida y la matriz de confusión. Esto nos permite comprobar si el modelo es capaz de generalizar a datos nuevos, no vistos en el entrenamiento.

El porcentaje de datos que se usan para el entrenamiento y el testeo influye en la exactitud de la red, por tanto, se harán varias pruebas con diferentes porcentajes para analizar cuál es mejor. Con un porcentaje muy alto de datos de entrenamiento podría darse un sobreajuste del modelo y con uno demasiado bajo podría no alcanzar su configuración más óptima.

4.5. Resultados

El porcentaje de datos utilizados para el testeo y para el entrenamiento puede variar los resultados de nuestro modelo ya que un porcentaje muy pequeño de entrenamiento podría empeorar el aprendizaje pero un porcentaje demasiado alto conllevaría un sobreajuste de la red. Por tanto, en primer lugar se hace un tanteo de cuáles son los porcentajes óptimos entre 10 y 30 % de testeo, hallando al final de cada entrenamiento la exactitud para poder compararlos. La exactitud o *accuracy* se halla según la Ec. (4.1).

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} = \frac{\text{Correct predictions}}{\text{All predictions}} \quad (4.1)$$

Siendo TP True Positive, TN True Negative, FP False Positive y FN False Negative.

Como se puede ver en la *tabla* 4.1, aunque los valores no difieren mucho, el porcentaje con el mejor resultado es 25 % de datos de testeo, por lo que el resto de las simulaciones las haremos con este parámetro.

Porcentaje de testeo	Accuracy
10 %	0.9797
15 %	0.9761
20 %	0.9761
25 %	0.9807
30 %	0.9774

Tabla 4.1: *Accuracies* para diferentes porcentajes de testeo.

Se hicieron varias simulaciones con conjuntos de entrenamiento y de testeo elegidos al azar y la exactitud media obtenida fue de $(97,88 \pm 0,13) \%$, con un tiempo de computación aproximado de 39 minutos con un procesador Core i5 de décima generación. A continuación se representa la curva de exactitud frente a las épocas, en la que se observa la evolución de la red durante el entrenamiento.

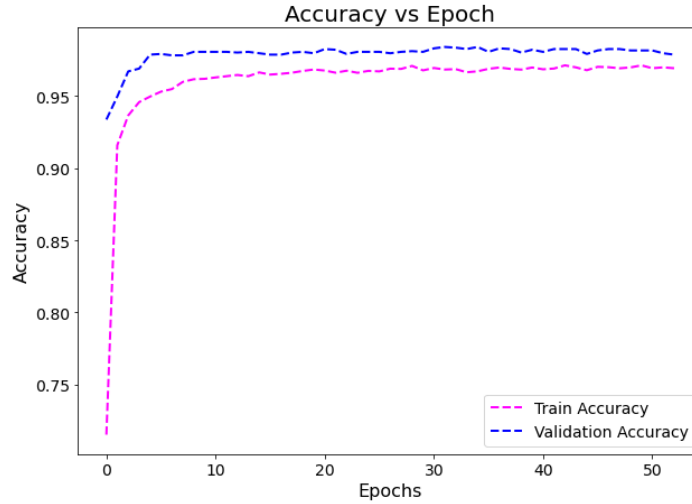


Figura 4.3: Curva de exactitud frente a las épocas para el conjunto de entrenamiento y el de validación.

Para terminar de visualizar el desempeño de modelo se representa la matriz de confusión (Figura 4.4). Es una herramienta muy utilizada en aprendizaje supervisado en problemas de clasificación ya que permite ver concretamente dónde están los fallos de clasificación. Las columnas representan las predicciones y las filas la clase real. Por tanto, los valores de la diagonal corresponden a la cantidad de aciertos sobre el conjunto de testeo, y las celdas fuera de la diagonal al número de imágenes que no fueron clasificadas correctamente.

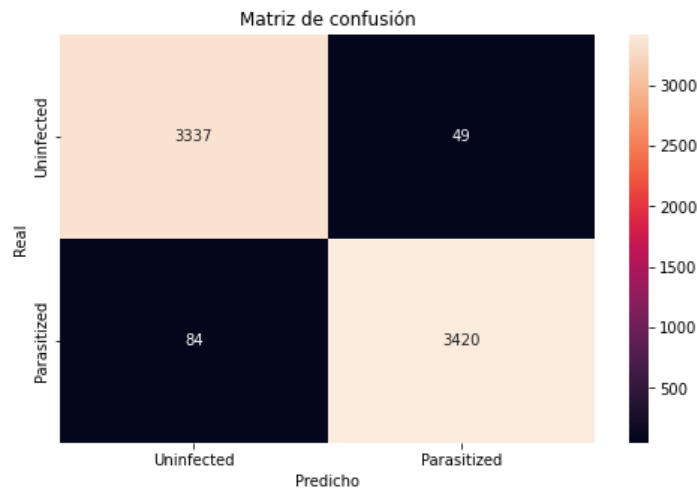


Figura 4.4: Matriz de confusión para un 25% de datos de testeo y un 75% de datos de entrenamiento.

4.6. Conclusiones

Fijándonos primero en la Figura 4.3, la exactitud de validación es siempre mayor que la de entrenamiento, por lo que el modelo no se sobreajusta al conjunto de entrenamiento. Además,

una exactitud del $(97,88 \pm 0,13) \%$ indica que el modelo es muy bueno, lo cual se debe entre otras cosas a la gran cantidad de datos y calidad de los mismos, así como a la buena elección de la arquitectura de la red para este problema. Es un *dataset* interesante para hacer una aplicación de redes convolucionales y obtener un buen resultado.

Siguiendo esta línea, en la matriz de confusión (*Figura 4.4*) se observa que los valores de la diagonal son mucho mayores que los de fuera de la diagonal, siendo el error de clasificación casi el doble para las imágenes infectadas de malaria, que se han clasificado erróneamente 84 veces frente a 49 en el caso de las no infectadas. Esto puede ser debido a que lo que diferencia a las imágenes de las células infectadas de las no infectadas es que se ve el parásito, y puede ser que si el parásito era muy pequeño, con el método de data augmentation o durante el entrenamiento se halla perdido esa información.

Este modelo ha sido diseñado después de varias pruebas con diferentes combinaciones de capas e hiperparámetros, pero la obtención de un modelo aún mejor requeriría hacer un análisis más extensivo explorando de estas variables, así como probar a implementar otras técnicas de regularización o obtener más datos.

Bibliografía

- [1] D. Hubel and T. Wiesel, “Receptive fields of single neurones in the cat’s striate cortex,” *The Journal of Physiology*, vol. 124(3), p. 574–591, 1959.
- [2] C. Aggarwal, *Neural Networks and Deep Learning, a Textbook*. Springer International Publishing AG, 2018.
- [3] P. Marimuthu, “How activation functions work in deep learning,” 2022, disponible en: <https://www.kdnuggets.com/2022/06/activation-functions-work-deep-learning.html>.
- [4] B. Kröse and P. van der Smagt, *An introduction to neural networks*. University of Amsterdam, Enero 1996, disponible en: https://www.researchgate.net/publication/225067619_An_Introduction_to_Neural_Networks.
- [5] E. Pekel and S. Kara, “A comprehensive review for artificial neural network application to public transportation,” *Sigma Journal of Engineering and Natural Sciences*, vol. 35, pp. 157–179, 03 2017.
- [6] J. Escobar, “Función de pérdida en el entrenamiento de una red neuronal,” 2024, disponible en: <https://iartificial.blog/aprendizaje/machine-learning/funcion-de-perdida-en-el-entrenamiento-de-una-red-neuronal/>.
- [7] “Descenso del gradiente,” 2024, disponible en: https://es.wikipedia.org/wiki/Descenso_del_gradiente.
- [8] P. Almagro, “Algoritmo de retropropagación,” Master’s thesis, Universidad de Sevilla, 2016.
- [9] Y. B. I. Goodfellow and A. Courville, *Deep learning*. MIT press, 2016, disponible en: <http://www.deeplearningbook.org/contents/convnets.html>.
- [10] OMS, “Informe mundial sobre la malaria 2022 - preguntas y respuestas con el dr abdisalan noor,” 2022, disponible en: <https://www.who.int/es/teams/global-malaria-programme/reports/world-malaria-report-2022/questions-and-answers>.
- [11] “Malaria screener datasheet,” disponible en: <https://lhncbc.nlm.nih.gov/LHC-downloads/downloads.html#malaria-datasets>.
- [12] Arunava, “Malaria cell images dataset,” 2018, disponible en: <https://www.kaggle.com/datasets/iarunava/cell-images-for-detecting-malaria>.

Anexos

Anexos A

Código Fuente

En este anexo se proporciona el código de Python elaborado para la aplicación práctica del Capítulo 4 con algunas explicaciones para facilitar su comprensión. No se han puesto tildes debido a una incompatibilidad con el programa de redacción del documento.

```
1 import os
2
3 from PIL import Image
4
5 import numpy as np
6
7 import pandas as pd
8
9 import matplotlib.pyplot as plt
10
11 import seaborn as sns
12
13 from sklearn.model_selection import train_test_split
14
15 from sklearn.preprocessing import MinMaxScaler
16
17 import tensorflow as tf
18
19 from tensorflow.keras import optimizers
20
21 import warnings
22
23 warnings.filterwarnings('ignore')
24
25 pd.set_option("display.max_columns", None)
26 pd.set_option("display.max_rows", 200)
27
28 cwd = os.getcwd() # Para ver en que directorio estamos
29 files = os.listdir(cwd) # Nos dice todos los archivos que hay en el directorio
30 print("Files in %r: %s" % (cwd, files))
31
32 #%% Importamos las librerías requeridas para construir y entrenar el modelo
33 from tensorflow.keras import backend
```

```

34
35 from tensorflow.keras.utils import to_categorical
36
37 from tensorflow.keras.models import Sequential
38
39 from keras.layers import Conv2D, MaxPooling2D, Dropout, Flatten, Dense,
40 BatchNormalization, LeakyReLU
41
42 from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
43
44 from random import shuffle
45 from sklearn.metrics import classification_report
46
47 from sklearn.metrics import confusion_matrix
48
49 backend.clear_session()
50
51 # Fijamos la semilla para la generacion de numeros aleatorios,
52 # asi nos aseguramos de que recibimos la misma salida cada vez
53 np.random.seed(42)
54
55 import random
56
57 random.seed(42)
58
59 tf.random.set_seed(42)
60 %% Vamos a obtener las imagenes para el entrenamiento
61 all_dir = r"C:\Users\olaya\OneDrive\Documentos\Uni\4o\2o_cuatri\TFG\codigo\DeepLearning-Ma
62
63 if not os.path.exists(all_dir):
64     print(f"Error: El directorio {all_dir} no existe.")
65
66 SIZE = 64
67 all_images = []
68 all_labels = []
69
70 for folder_name in ['parasitized', 'uninfected']:
71
72     folder_path = os.path.join(all_dir, folder_name)
73     if not os.path.exists(folder_path):
74         print(f"Error: El directorio {folder_path} no existe.")
75         continue
76
77     #se obtiene la lista con los nombres de las imagenes en la carpeta actual
78     images_path = os.listdir(folder_path)
79
80
81     for i, image_name in enumerate(images_path):
82
83         try:
84             image_path = os.path.join(folder_path, image_name)
85
86             # Abrimos cada imagen utilizando su ruta
87             image = Image.open(image_path)
88

```

```

89         # Redimensionamos cada imagen a (64, 64)
90         image = image.resize((SIZE, SIZE))
91
92         # Convertimos las im genes a arrays y lo a adimos a la lista
93         all_images.append(np.array(image))
94
95         # Creaamos las etiquetas parasitized y uninfected para las im genes
96         if folder_name == 'parasitized':
97             all_labels.append(1)
98
99         else:
100             all_labels.append(0)
101
102     except Exception as e:
103         print(f"Error al procesar {image_path}: {e}")
104         pass
105
106 # Convertimos las listas a arrays
107 all_images = np.array(all_images)
108 all_labels = np.array(all_labels)
109
110 # Verificamos las dimensiones de los arrays
111 print(f"Dimensiones de train_images: {all_images.shape}")
112 print(f"Dimensiones de train_labels: {all_labels.shape}")
113
114 ###
115 # Verificamos el rango minimo y maximo de pixeles
116 all_min_pixel = np.min(all_images)
117 all_max_pixel = np.max(all_images)
118 print(f"Rango minimo de pixeles en all_images: {all_min_pixel}")
119 print(f"Rango maximo de pixeles en all_images: {all_max_pixel}")
120
121 #Contamos cuantas imagenes son uninfected y parasitized
122 print(f"Numero de etiquetas 'uninfected': {np.count_nonzero(all_labels == 0)}")
123 print(f"Numero de etiquetas 'parasitized': {np.count_nonzero(all_labels == 1)}")
124
125 #NORMALIZAMOS train and test imagenes dividiendolas por 255 o convirtiendolas
126 en float32
127 all_images = all_images.astype('float32') / 255.0
128
129 all_max_pixel = np.max(all_images)
130 print(f"Rango maximo de p xeles en train_images: {all_max_pixel}")
131
132 ### Convertimos las etiquetas a formato categorico
133
134 all_labels = to_categorical(all_labels, 2)
135 ###
136 # Dividimos en conjunto de entrenamiento y conjunto de prueba
137 #Con esta funcion los datos se mezclan aleatoriamente antes de dividirse
138 X_train_val, test_images, y_train_val, test_labels = train_test_split(all_images,
139 all_labels, test_size=0.25, random_state=42)
140
141 # Ahora dividimos el conjunto de entrenamiento en entrenamiento
142 #y validacion
143 X_train, X_val, y_train, y_val = train_test_split(X_train_val, y_train_val,

```

```

144 test_size=0.1, random_state=42)
145
146 ### Implementamos data augmentation
147
148 from tensorflow.keras.preprocessing.image import ImageDataGenerator
149
150 # Usamos ImageDataGenerator para generar imagenes
151 train_datagen = ImageDataGenerator(horizontal_flip = True,
152                                     zoom_range = [0.7,1.0], rotation_range = 30)
153
154 val_datagen = ImageDataGenerator()
155
156 # Generamos lotes de datos de las imagenes y las etiquetas correspondientes,
157 # utilizando el generador que acabamos de definir
158 train_generator = train_datagen.flow(x = X_train, y=y_train, batch_size = 64,
159                                     seed = 42, shuffle = True)
160
161 val_generator = val_datagen.flow(x=X_val, y=y_val, batch_size = 64, seed = 42,
162                                 shuffle = True)
163
164 ### Vamos a plotear unas celulas y algunas de sus posibles transformaciones
165 # Seleccionamos algunas imagenes originales
166 num_images = 4
167 original_images = X_train[:num_images]
168 original_labels = y_train[:num_images]
169
170 # Creamos un lote de imagenes aumentadas manualmente
171 augmented_images = np.array([train_datagen.random_transform(image) for
172 image in original_images])
173
174 fig, axes = plt.subplots(2, num_images, figsize=(16, 8))
175 fig.set_size_inches(16, 8)
176
177 # Ploteamos las imagenes originales en la primera fila
178 for i in range(num_images):
179     ax = axes[0, i]
180     ax.imshow(original_images[i])
181
182     if original_labels[i][1] == 1:
183         ax.set_title('Original - Parasitized')
184     else:
185         ax.set_title('Original - Uninfected')
186
187     ax.axis('off')
188
189 # Ploteamos las imagenes aumentadas en la segunda fila
190 for i in range(num_images):
191     ax = axes[1, i]
192     ax.imshow(augmented_images[i])
193
194     if original_labels[i][1] == 1:
195         ax.set_title('Transformada - Parasitized')
196     else:
197         ax.set_title('Transformada - Uninfected')
198

```

```

199     ax.axis('off')
200
201 plt.show()
202 ### Construimos nuestra red
203
204 backend.clear_session()
205 model3 = Sequential()
206
207 model3.add(Conv2D(filters = 32, kernel_size = 2, padding = "same",
208                  activation = "relu", input_shape = (64, 64, 3)))
209 model3.add(MaxPooling2D(pool_size = 2))
210 model3.add(Dropout(0.2))
211
212 model3.add(Conv2D(filters = 32, kernel_size = 2, padding = "same",
213                  activation = "relu"))
214 model3.add(MaxPooling2D(pool_size = 2))
215 model3.add(Dropout(0.2))
216
217 model3.add(Conv2D(filters = 32, kernel_size = 2, padding = "same",
218                  activation = "relu"))
219 model3.add(MaxPooling2D(pool_size = 2))
220 model3.add(Dropout(0.2))
221
222 model3.add(Flatten())
223
224 model3.add(Dense(512, activation = "relu"))
225 model3.add(Dropout(0.4))
226
227 # Aqui 2 representa las neuronas de las capas de salida
228 model3.add(Dense(2, activation = "softmax"))
229
230 # Mostramos un resumen del modelo
231 model3.summary()
232
233 ###
234 model3.compile(loss='binary_crossentropy', optimizer="adam", metrics=['accuracy'])
235
236 callbacks = [EarlyStopping(monitor='val_loss', patience=10),
237             ModelCheckpoint('.mdl_wts.keras', monitor='val_loss', save_best_only=True)]
238
239 ###
240 history3 = model3.fit(train_generator, validation_data = val_generator,
241                     batch_size = 32, callbacks = callbacks, epochs = 70, verbose = 1)
242 ###
243
244 accuracy3 = model3.evaluate(test_images, test_labels, verbose=1)
245 print('\n', 'Test_Accuracy_3:-', accuracy3[1])
246
247 ###
248 print(history3.history.keys())
249 print("Epochs trained:", len(history3.history['accuracy']))

```

Listing A.1: Código empleado en el tratamiento de datos y en la red convolucional.

Anexos B

Entrenamiento de la CNN

En este anexo se recoge el resumen del entrenamiento en una de las simulaciones para un 25 % de datos utilizados para el testeo.

```
1 Epoch 1/70
2 291/291    69s 220ms/step - accuracy: 0.5349 - loss: 0.6942 -
3 val_accuracy: 0.5481 - val_loss: 0.6598
4 Epoch 2/70
5 291/291    43s 147ms/step - accuracy: 0.7293 - loss: 0.5512 -
6 val_accuracy: 0.9468 - val_loss: 0.1804
7 Epoch 3/70
8 291/291    43s 146ms/step - accuracy: 0.9211 - loss: 0.2182 -
9 val_accuracy: 0.9690 - val_loss: 0.1195
10 Epoch 4/70
11 291/291    43s 145ms/step - accuracy: 0.9428 - loss: 0.1745 -
12 val_accuracy: 0.9734 - val_loss: 0.1013
13 Epoch 5/70
14 291/291    47s 159ms/step - accuracy: 0.9508 - loss: 0.1512 -
15 val_accuracy: 0.9768 - val_loss: 0.0997
16 Epoch 6/70
17 291/291    45s 151ms/step - accuracy: 0.9522 - loss: 0.1427 -
18 val_accuracy: 0.9758 - val_loss: 0.0863
19 Epoch 7/70
20 291/291    57s 192ms/step - accuracy: 0.9592 - loss: 0.1325 -
21 val_accuracy: 0.9763 - val_loss: 0.0888
22 Epoch 8/70
23 291/291    38s 129ms/step - accuracy: 0.9616 - loss: 0.1167 -
24 val_accuracy: 0.9787 - val_loss: 0.0733
25 Epoch 9/70
26 291/291    35s 117ms/step - accuracy: 0.9586 - loss: 0.1215 -
27 val_accuracy: 0.9777 - val_loss: 0.0710
28 Epoch 10/70
29 291/291    35s 117ms/step - accuracy: 0.9628 - loss: 0.1132 -
30 val_accuracy: 0.9806 - val_loss: 0.0662
31 Epoch 11/70
32 291/291    40s 134ms/step - accuracy: 0.9637 - loss: 0.1114 -
33 val_accuracy: 0.9821 - val_loss: 0.0665
34 Epoch 12/70
35 291/291    35s 118ms/step - accuracy: 0.9655 - loss: 0.1070 -
```

```

36 val_accuracy: 0.9782 - val_loss: 0.0785
37 Epoch 13/70
38 291/291   33s 111ms/step - accuracy: 0.9652 - loss: 0.1113 -
39 val_accuracy: 0.9792 - val_loss: 0.0642
40 Epoch 14/70
41 291/291   35s 118ms/step - accuracy: 0.9652 - loss: 0.1031 -
42 val_accuracy: 0.9792 - val_loss: 0.0618
43 Epoch 15/70
44 291/291   36s 120ms/step - accuracy: 0.9639 - loss: 0.1097 -
45 val_accuracy: 0.9816 - val_loss: 0.0582
46 Epoch 16/70
47 291/291   33s 111ms/step - accuracy: 0.9642 - loss: 0.1064 -
48 val_accuracy: 0.9811 - val_loss: 0.0616
49 Epoch 17/70
50 291/291   35s 118ms/step - accuracy: 0.9668 - loss: 0.1028 -
51 val_accuracy: 0.9806 - val_loss: 0.0596
52 Epoch 18/70
53 291/291   40s 134ms/step - accuracy: 0.9655 - loss: 0.1030 -
54 val_accuracy: 0.9792 - val_loss: 0.0584
55 Epoch 19/70
56 291/291   43s 147ms/step - accuracy: 0.9690 - loss: 0.0939 -
57 val_accuracy: 0.9816 - val_loss: 0.0586
58 Epoch 20/70
59 291/291   40s 135ms/step - accuracy: 0.9663 - loss: 0.1058 -
60 val_accuracy: 0.9797 - val_loss: 0.0582
61 Epoch 21/70
62 291/291   40s 134ms/step - accuracy: 0.9654 - loss: 0.1028 -
63 val_accuracy: 0.9802 - val_loss: 0.0593
64 Epoch 22/70
65 291/291   33s 110ms/step - accuracy: 0.9653 - loss: 0.1050 -
66 val_accuracy: 0.9806 - val_loss: 0.0611
67 Epoch 23/70
68 291/291   33s 112ms/step - accuracy: 0.9688 - loss: 0.1002 -
69 val_accuracy: 0.9826 - val_loss: 0.0547
70 Epoch 24/70
71 291/291   35s 117ms/step - accuracy: 0.9681 - loss: 0.1004 -
72 val_accuracy: 0.9831 - val_loss: 0.0548
73 Epoch 25/70
74 291/291   38s 129ms/step - accuracy: 0.9686 - loss: 0.0944 -
75 val_accuracy: 0.9773 - val_loss: 0.0634
76 Epoch 26/70
77 291/291   38s 129ms/step - accuracy: 0.9698 - loss: 0.0921 -
78 val_accuracy: 0.9787 - val_loss: 0.0642
79 Epoch 27/70
80 291/291   46s 156ms/step - accuracy: 0.9696 - loss: 0.0960 -
81 val_accuracy: 0.9802 - val_loss: 0.0538
82 Epoch 28/70
83 291/291   38s 129ms/step - accuracy: 0.9681 - loss: 0.0948 -
84 val_accuracy: 0.9806 - val_loss: 0.0535
85 Epoch 29/70
86 291/291   44s 149ms/step - accuracy: 0.9707 - loss: 0.0895 -
87 val_accuracy: 0.9836 - val_loss: 0.0498
88 Epoch 30/70
89 291/291   38s 130ms/step - accuracy: 0.9679 - loss: 0.0953 -
90 val_accuracy: 0.9836 - val_loss: 0.0529

```

```

91 Epoch 31/70
92 291/291   38s 129ms/step - accuracy: 0.9680 - loss: 0.0916 -
93 val_accuracy: 0.9821 - val_loss: 0.0524
94 Epoch 32/70
95 291/291   32s 108ms/step - accuracy: 0.9686 - loss: 0.0958 -
96 val_accuracy: 0.9840 - val_loss: 0.0508
97 Epoch 33/70
98 291/291   34s 115ms/step - accuracy: 0.9680 - loss: 0.0960 -
99 val_accuracy: 0.9826 - val_loss: 0.0523
100 Epoch 34/70
101 291/291   32s 107ms/step - accuracy: 0.9693 - loss: 0.0866 -
102 val_accuracy: 0.9836 - val_loss: 0.0527
103 Epoch 35/70
104 291/291   37s 125ms/step - accuracy: 0.9677 - loss: 0.1014 -
105 val_accuracy: 0.9802 - val_loss: 0.0555
106 Epoch 36/70
107 291/291   36s 121ms/step - accuracy: 0.9689 - loss: 0.0937 -
108 val_accuracy: 0.9840 - val_loss: 0.0498
109 Epoch 37/70
110 291/291   33s 112ms/step - accuracy: 0.9706 - loss: 0.0895 -
111 val_accuracy: 0.9826 - val_loss: 0.0547
112 Epoch 38/70
113 291/291   34s 115ms/step - accuracy: 0.9684 - loss: 0.0947 -
114 val_accuracy: 0.9821 - val_loss: 0.0491
115 Epoch 39/70
116 291/291   32s 108ms/step - accuracy: 0.9705 - loss: 0.0928 -
117 val_accuracy: 0.9821 - val_loss: 0.0482
118 Epoch 40/70
119 291/291   35s 118ms/step - accuracy: 0.9702 - loss: 0.0888 -
120 val_accuracy: 0.9806 - val_loss: 0.0499
121 Epoch 41/70
122 291/291   32s 109ms/step - accuracy: 0.9712 - loss: 0.0892 -
123 val_accuracy: 0.9811 - val_loss: 0.0508
124 Epoch 42/70
125 291/291   32s 108ms/step - accuracy: 0.9669 - loss: 0.0978 -
126 val_accuracy: 0.9845 - val_loss: 0.0486
127 Epoch 43/70
128 291/291   40s 135ms/step - accuracy: 0.9687 - loss: 0.0938 -
129 val_accuracy: 0.9826 - val_loss: 0.0508
130 Epoch 44/70
131 291/291   34s 114ms/step - accuracy: 0.9701 - loss: 0.0881 -
132 val_accuracy: 0.9816 - val_loss: 0.0634
133 Epoch 45/70
134 291/291   32s 109ms/step - accuracy: 0.9707 - loss: 0.0897 -
135 val_accuracy: 0.9816 - val_loss: 0.0545
136 Epoch 46/70
137 291/291   38s 127ms/step - accuracy: 0.9699 - loss: 0.0896 -
138 val_accuracy: 0.9806 - val_loss: 0.0531
139 Epoch 47/70
140 291/291   32s 106ms/step - accuracy: 0.9716 - loss: 0.0857 -
141 val_accuracy: 0.9845 - val_loss: 0.0494
142 Epoch 48/70
143 291/291   33s 112ms/step - accuracy: 0.9665 - loss: 0.0941 -
144 val_accuracy: 0.9811 - val_loss: 0.0524
145 Epoch 49/70

```



```

146 291/291    39s 131ms/step - accuracy: 0.9730 - loss: 0.0863 -
147 val_accuracy: 0.9831 - val_loss: 0.0468
148 Epoch 50/70
149 291/291    33s 110ms/step - accuracy: 0.9716 - loss: 0.0864 -
150 val_accuracy: 0.9840 - val_loss: 0.0483
151 Epoch 51/70
152 291/291    31s 105ms/step - accuracy: 0.9727 - loss: 0.0821 -
153 val_accuracy: 0.9831 - val_loss: 0.0496
154 Epoch 52/70
155 291/291    32s 107ms/step - accuracy: 0.9688 - loss: 0.0904 -
156 val_accuracy: 0.9821 - val_loss: 0.0500
157 Epoch 53/70
158 291/291    32s 107ms/step - accuracy: 0.9680 - loss: 0.0929 -
159 val_accuracy: 0.9831 - val_loss: 0.0515
160 Epoch 54/70
161 291/291    34s 116ms/step - accuracy: 0.9717 - loss: 0.0840 -
162 val_accuracy: 0.9821 - val_loss: 0.0511
163 Epoch 55/70
164 291/291    30s 102ms/step - accuracy: 0.9724 - loss: 0.0836 -
165 val_accuracy: 0.9831 - val_loss: 0.0497
166 Epoch 56/70
167 291/291    31s 105ms/step - accuracy: 0.9703 - loss: 0.0867 -
168 val_accuracy: 0.9816 - val_loss: 0.0551
169 Epoch 57/70
170 291/291    31s 103ms/step - accuracy: 0.9736 - loss: 0.0802 -
171 val_accuracy: 0.9821 - val_loss: 0.0502
172 Epoch 58/70
173 291/291    31s 104ms/step - accuracy: 0.9668 - loss: 0.1032 -
174 val_accuracy: 0.9836 - val_loss: 0.0534
175 Epoch 59/70
176 291/291    33s 110ms/step - accuracy: 0.9696 - loss: 0.0902 -
177 val_accuracy: 0.9855 - val_loss: 0.0471
178
179 Test_Accuracy_3:- 0.9806966781616211

```

Listing B.1: Resumen del entrenamiento de la red convolucional.